Docker Transport Lab Manual

This manual documents step-by-step instructions to set up Docker and implement four transport mechanisms—REST (HTTP/JSON), MQTT and gRPC — on Ubuntu OS.

Prerequisites

A Linux machine Ubuntu. A user with sudo privileges. Internet access to download packages and Docker images. This manual assumes basic familiarity with the terminal.

How exactly Docker is used?

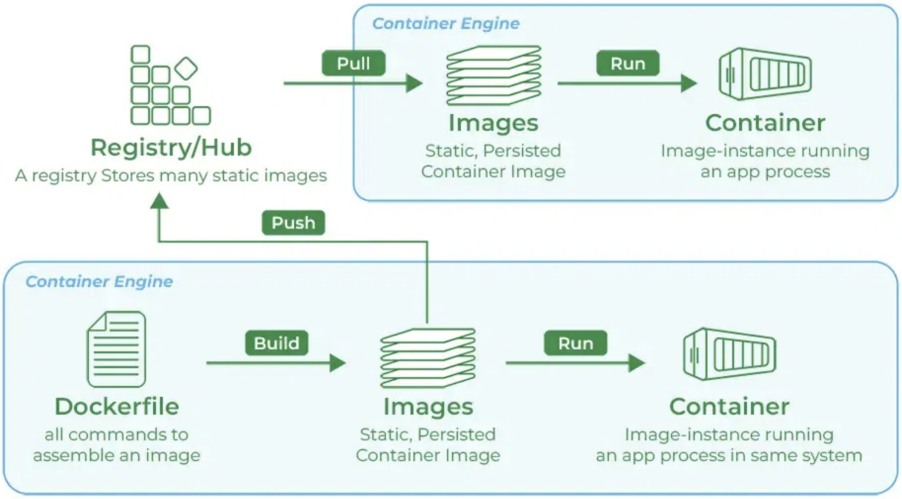

Figure 1: How Docker Work

What is Docker

Docker is a platform designed to help developers build, share, and run containerized applications. A container packages an application together with its runtime, libraries, and configuration so the same artifact runs consistently across different environments (development, testing, production).

Key benefits:

- **Portability:** an image runs the same on any host with Docker installed.
- **Isolation:** processes, file systems and networks are isolated per container.
- **Reproducibility:** images are built from versioned Dockerfiles.
- **Efficiency:** containers are lighter than virtual machines and start quickly.

Why do we need dockers?

1 Consistency Across Environments

- **Problem:** Applications often behave differently in development,testing, and production environments due to variations in configurations, dependencies, and infrastructure.

- **Solution:** Docker containers encapsulate all the necessary components, ensuring the application runs consistently across all environments.

2 Isolation

- **Problem:** Running multiple applications on the same host can lead to conflicts, such as dependency clashes or resource contention..
- **Solution:** Docker provides isolated environments for each application, preventing interference and ensuring stable performance.

3 Scalability

- **Problem:** Scaling applications to handle increased load can be challenging, requiring manual intervention and configuration.
- **Solution:** Docker makes it easy to scale applications horizontally by running multiple container instances, allowing for quick and efficient scaling.

Components of Docker

Docker Engine

Docker Engine is the core component of the Docker platform, responsible for creating,running, and managing Docker containers.

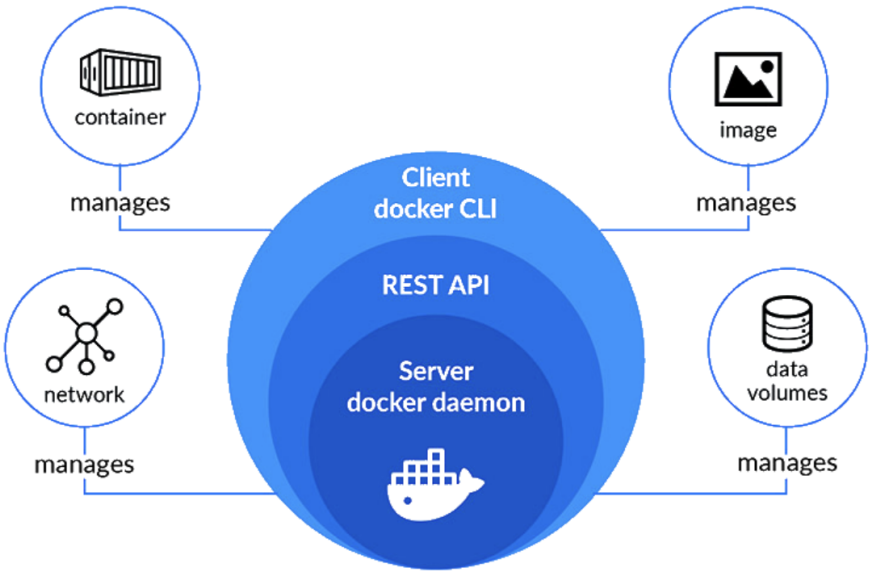

Figure 2: Docker Engine

Docker Image

A Docker image is a lightweight, stand-alone, and executable software package that includes everything needed to run a piece of software, such as the code, runtime, libraries,environment

variables, and configuration files. Images are used to create Docker containers, which are instances of these images.

Components of a Docker Image

- **Base Image:** The starting point for building an image. It could be a minimal OS image like alpine, a full-fledged OS like ubuntu, or even another application image like python or node.
- **Application Code:** The actual code and files necessary for the application to run.
- **Dependencies:**Libraries, frameworks, and packages required by the application.
- **Metadata:**Metadata: Information about the image, such as environment variables, labels, and exposed ports.

Dockerfile

A Dockerfile is a text file that contains a series of instructions used to build a Docker image.Each instruction in a Dockerfile creates a layer in the image, allowing for efficient image creation and reuse of layers. Dockerfiles are used to automate the image creation process,ensuring consistency and reproducibility.

Docker Container

A Docker container is a lightweight, portable, and isolated environment that encapsulates an application and its dependencies, allowing it to run consistently across different computing environments. Containers are created from Docker images, which are immutable and contain all the necessary components for the application to run.

Registry

A Docker registry is a service that stores and distributes Docker images. It acts as a repository where users can push, pull, and manage Docker images. Docker Hub is the most well-known public registry, but private registries can also be set up to securely store and manage images within an organization.

4 Install Docker

Use Docker's official convenience script (safe and widely used). The commands below assume a current Ubuntu release and a user with sudo privileges.

```
# Update system
sudo apt update
sudo apt full-upgrade -y

# Install Docker Engine (official convenience script)
curl -fsSL https://get.docker.com | sudo sh

# Add your user to the docker group (replace $USER as needed)
sudo usermod -aG docker $USER

# Apply the new group membership (log out/in or run)
```

```dockerfile
# Use an official Python runtime as a base image
FROM python:3.8-slim

# Set the working directory in the container
WORKDIR /app

# Copy the current directory contents into the container at /app
COPY . /app

# Install any needed packages specified in requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Make port 80 available to the world outside this container
EXPOSE 80

# Define environment variable
ENV NAME World

# Run app.py when the container launches
CMD ["python", "app.py"]
```

Figure 3: Dockerfile

```
newgrp docker

# Verify Docker installation
docker --version
docker run --rm hello-world

# Optional: enable Docker to start on boot
sudo systemctl enable --now docker
```

Notes:

- If your environment restricts remote downloads, install via the Ubuntu package repo or follow Docker's official APT repository instructions.
- On systems using 'sudo' for Docker commands, add your user to the "docker" group for convenience.

5 Quick Docker housekeeping and checks

Confirm architecture and running docker daemon:

```
uname -m
docker --version
```

```
docker ps
```

Basic Docker Examples

Run a quick container

```
# Run the official hello-world image
docker run --rm hello-world
```

Build and run a simple app

```
# Build image from Dockerfile in current directory
docker build -t myapp:latest .

# Run container, map host port 8080 to container port 80
docker run -d --name myapp -p 8080:80 myapp:latest

# Inspect containers and view logs
docker ps
docker logs myapp

# Stop and remove the container
docker stop myapp
docker rm myapp
```

Useful commands

- `docker images` — list images
- `docker rm -f <container>` — force remove a container
- `docker system prune -a` — cleanup unused images/containers

6 Transport Mechanisms: Overview

Transport mechanisms are the communication patterns and protocols used by distributed systems. Typical mechanisms used in cloud/IoT and microservice systems include:

- **REST (HTTP/JSON)** — stateless request/response over HTTP; human-readable and widely compatible.
- **gRPC** — RPC framework using Protocol Buffers over HTTP/2; binary, strongly typed, and high-performance.
- **MQTT** — lightweight publish/subscribe protocol for constrained devices and unreliable networks.

Each mechanism trades off readability, performance, feature set (e.g., streaming), and operational complexity. Select the mechanism according to latency, bandwidth, device constraints, and developer ecosystem.

7 REST (HTTP/JSON) — Step-by-step with nano

REST: Short Explanation

REST (Representational State Transfer) is an architectural style that uses HTTP methods (GET, POST, PUT, DELETE) to operate on resources. Payloads are typically JSON. REST is stateless; each request contains all information needed to process it. Use REST for public APIs, web services and easy-to-debug integrations.

7.1 REST Transport (HTTP) Implementation Using Docker

This subsection documents a complete step-by-step laboratory procedure to implement a RESTful service over HTTP using Python Flask and Docker on a Raspberry Pi. All files are created manually using the `nano` editor to ensure reproducibility and compliance with practical examination requirements.

7.1.1 Step 1: Project Directory Creation

First, create the project directory structure for the REST service.

```
mkdir -p ~/rest-service/app
cd ~/rest-service
```

This command creates a parent directory for the service and an `app` subdirectory to store application source files.

7.1.2 Step 2: REST Application Source File

Create the main REST application file using `nano`.

```
nano app/main.py
```

Insert the following code:

```
import os
from flask import Flask, jsonify, request

SERVICE_NAME = os.getenv("SERVICE_NAME", "rest-service")
SERVICE_VERSION = os.getenv("SERVICE_VERSION", "v1")

app = Flask(__name__)

def response(data=None, message="OK", status=200):
    return jsonify({
        "service": SERVICE_NAME,
        "version": SERVICE_VERSION,
        "message": message,
        "data": data
    }), status

@app.route("/health", methods=["GET"])
def health():
    return response({"status": "healthy"})

REQUEST_COUNT = 0

@app.before_request
def count_requests():
    global REQUEST_COUNT
    REQUEST_COUNT += 1

@app.route("/metrics", methods=["GET"])
```

```
def metrics():
    return response({
        "requests_total": REQUEST_COUNT
    })

@app.route("/api/v1/data", methods=["GET"])
def get_data():
    return response({
        "sensor": "heart_rate",
        "value": 72,
        "unit": "bpm"
    })

@app.route("/api/v1/data", methods=["POST"])
def post_data():
    if not request.is_json:
        return response(message="Invalid JSON payload", status=400)

    payload = request.get_json()

    if "sensor" not in payload or "value" not in payload:
        return response(message="Missing required fields", status=422)

    return response(
        data=payload,
        message="Data received",
        status=201
    )

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

This application exposes REST endpoints for health checks, metrics, and sensor data exchange using JSON over HTTP.

7.1.3 Step 3: Dependency Specification

Create the Python dependency file.

```
nano app/requirements.txt
```

Add:

```
flask==3.0.0
```

This file ensures consistent dependency installation inside the container.

7.1.4 Step 4: Dockerfile Creation

Create the Dockerfile to containerize the REST service.

```
nano Dockerfile
```

Insert:

```
FROM python:3.10-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

EXPOSE 5000

CMD ["python", "main.py"]
```

This Dockerfile defines a lightweight Python runtime, installs dependencies, and exposes the REST service on port 5000.

7.1.5 Step 5: Docker Image Build

Build the Docker image locally.

```
docker build -t rest-advanced .
```

This step packages the REST application into a reusable container image.

7.1.6 Step 6: Container Execution

Run the REST service as a Docker container.

```
docker run -d \
  --name rest-advanced \
  -p 5000:5000 \
  -e SERVICE_NAME="REST-LAB" \
  -e SERVICE_VERSION="v1" \
  rest-advanced
```

Environment variables are used to configure service metadata at runtime.

7.1.7 Step 7: REST Endpoint Testing

Verify correct service operation using `curl`.

Health check:

```
curl http://localhost:5000/health
```

Metrics endpoint:

```
curl http://localhost:5000/metrics
```

GET request:

```
curl http://localhost:5000/api/v1/data
```

POST request:

```
curl -X POST http://localhost:5000/api/v1/data \
  -H "Content-Type: application/json" \
  -d '{"sensor":"temperature","value":36.5}'
```

These commands demonstrate RESTful request–response communication using HTTP methods.

7.1.8 Step 8: Log Inspection

View runtime logs for debugging and demonstration.

```
docker logs rest-advanced
```

7.1.9 Step 9: Container Cleanup

Stop and remove the container when testing is complete.

```
docker stop rest-advanced
docker rm rest-advanced
```

7.1.10 Conceptual Discussion

This experiment demonstrates:

- REST communication over TCP using HTTP
- Stateless request–response architecture
- JSON-based data exchange
- Service isolation using Docker containers
- Ease of debugging and deployment in REST systems

8 gRPC Transport Implementation Using Docker

gRPC: Short Explanation

gRPC is a Remote Procedure Call (RPC) framework that uses Protocol Buffers for binary serialization and HTTP/2 for transport. It supports unary and streaming RPCs, strongly-typed interfaces (.proto files), and typically provides lower latency and smaller payloads than JSON-over-HTTP. gRPC is well-suited for internal microservice communication and high-performance service-to-service calls.

This section documents the step-by-step implementation and deployment of a simple gRPC-based client–server application using Python, Protocol Buffers, and Docker containers on Ubuntu. The experiment demonstrates high-performance Remote Procedure Call (RPC) communication suitable for microservices and distributed systems.

8.1 Aim

To implement and deploy a gRPC client–server application using Docker containers.

8.2 Software Requirements

- Ubuntu (VMware or bare-metal installation)
- Docker Engine
- Python 3
- gRPC
- Protocol Buffers
- Docker Compose (via Python module)

8.3 Step 1: Project Directory Structure

Create the project directories for gRPC development.

```
mkdir grpc-lab
cd grpc-lab
mkdir proto server client
```

This structure separates protocol definitions, server implementation, and client implementation.

8.4 Step 2: Protocol Buffer Definition

Create the Protocol Buffer file defining the service and messages.

```
nano proto/hello.proto
```

Insert the following content:

```
syntax = "proto3";

service HelloService {
  rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest {
  string name = 1;
}

message HelloReply {
  string message = 1;
}
```

This file defines the gRPC service, RPC method, and request–response message formats.

8.5 Step 3: Install gRPC Tools on Host System

Install the required Python packages for gRPC code generation.

```
pip3 install grpcio grpcio-tools --break-system-packages
```

These tools are used to generate Python bindings from the Protocol Buffer file.

8.6 Step 4: Generate gRPC Python Files

Generate Python source files from the `.proto` definition.

```
python3 -m grpc_tools.protoc \
-I proto \
--python_out=. \
--grpc_python_out=. \
proto/hello.proto
```

Generated files:

- `hello_pb2.py`
- `hello_pb2_grpc.py`

Move the generated files to the server and client directories.

```
mv hello_pb2*.py server/
cp server/hello_pb2*.py client/
```

8.7 Step 5: gRPC Server Implementation

Create the gRPC server source file.

```
nano server/server.py
```

Insert the following code:

```
from concurrent import futures
import grpc
import hello_pb2
import hello_pb2_grpc

class HelloService(hello_pb2_grpc.HelloServiceServicer):
    def SayHello(self, request, context):
        return hello_pb2.HelloReply(
            message=f"Hello {request.name}, from gRPC Server!"
        )

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    hello_pb2_grpc.add_HelloServiceServicer_to_server(
        HelloService(), server
    )
    server.add_insecure_port('[::]:50051')
    server.start()
    server.wait_for_termination()

if __name__ == "__main__":
    serve()
```

This server listens on port 50051 and responds to client RPC calls.

8.8 Step 6: gRPC Client Implementation

Create the client application.

```
nano client/client.py
```

Insert:

```
import grpc
import hello_pb2
import hello_pb2_grpc

channel = grpc.insecure_channel('server:50051')
stub = hello_pb2_grpc.HelloServiceStub(channel)

response = stub.SayHello(
    hello_pb2.HelloRequest(name="Ubuntu LAB")
)

print("Server Response:", response.message)
```

The client invokes the remote procedure as a local function call.

8.9 Step 7: Dockerfile for gRPC Server

Create the Dockerfile for the server container.

```
nano server/Dockerfile
```

Insert:

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install grpcio protobuf
CMD ["python", "server.py"]
```

8.10 Step 8: Dockerfile for gRPC Client

Create the Dockerfile for the client container.

```
nano client/Dockerfile
```

Insert:

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install grpcio protobuf
CMD ["python", "client.py"]
```

8.11 Step 9: Docker Compose Configuration

Create the Docker Compose configuration file.

```
nano docker-compose.yml
```

Insert:

```
version: '3'
services:
  server:
    build: ./server
    ports:
      - "50051:50051"

  client:
    build: ./client
    depends_on:
      - server
```

Docker Compose enables multi-container orchestration for client–server communication.

8.12 Step 10: Run gRPC Application Using Docker Compose

Clean existing containers and images.

```
sudo python3 -m compose down --remove-orphans --volumes
sudo docker system prune -a -f
```

Build and run the application:

```
sudo python3 -m compose up --build
```

8.13 Output

```
Server Response: Hello Ubuntu LAB, from gRPC Server!
```

This confirms successful gRPC client–server communication.

8.14 Conceptual Discussion

- gRPC uses HTTP/2 over TCP for transport
- Protocol Buffers enable compact binary serialization
- gRPC supports strongly typed APIs and bidirectional streaming
- Docker ensures environment consistency and isolation
- gRPC provides higher performance than REST for microservices

9 MQTT: Publish/Subscribe Transport

Short Explanation

MQTT (Message Queuing Telemetry Transport) is a lightweight publish/subscribe messaging protocol optimized for constrained devices and unreliable networks. Communication is brokered: clients connect to a central broker, publish to topics, and subscribe to topics to receive messages.

Core concepts

- **Broker:** central server that routes messages (e.g., Eclipse Mosquitto).
- **Client:** publisher or subscriber connecting to the broker.
- **Topic:** hierarchical label for messages, e.g., `sensors/temperature`.
- **QoS (Quality of Service):** levels 0 (at most once), 1 (at least once), 2 (exactly once).

When to choose MQTT

Choose MQTT for telemetry, sensor networks, and low-bandwidth or high-latency environments.

Assignment 1 — Terminal MQTT (Local Broker)

Goal: Demonstrate publish/subscribe using a local Mosquitto broker and command-line tools.

1. Install Mosquitto and clients:

   ```
   sudo apt update
   sudo apt install -y mosquitto mosquitto-clients
   sudo systemctl enable --now mosquitto
   ss -ltn | grep 1883
   ```

2. In Terminal A (subscribe):

   ```
   mosquitto_sub -h localhost -t 'lab/topic' -v
   ```

3. In Terminal B (publish):

   ```
   mosquitto_pub -h localhost -t 'lab/topic' -m 'Hello from terminal'
   ```

4. Experiment with QoS:

   ```
   mosquitto_pub -h localhost -t 'lab/topic' -m 'QoS1' -q 1
     mosquitto_sub -h localhost -t 'lab/topic' -q 1 -v
   ```

5. Deliverable: one-page report with terminal screenshots demonstrating QoS 0 and QoS 1.

Assignment 2 — MQTT in Docker

Goal: Run an MQTT broker in Docker and publish messages from a containerized Python client.

Option A — Quick run with the Mosquitto image

```
docker run -d --name mqtt-broker -p 1883:1883 eclipse-mosquitto
mosquitto_sub -h localhost -t 'lab/topic' -v
mosquitto_pub -h localhost -t 'lab/topic' -m 'Hello from Docker'
```

Option B — Docker Compose (broker + publisher) Create `docker-compose.yml` and `mqtt-samples/pub.py` as follows:

```
version: '3'
services:
  broker:
    image: eclipse-mosquitto
    ports:
      - "1883:1883"

  publisher:
    image: python:3.10-slim
    command: ["/bin/sh","-c","pip install paho-mqtt && python /app/pub.py"]
    volumes:
      - ./mqtt-samples:/app
    depends_on:
      - broker
```

`mqtt-samples/pub.py`:

```
import time
import paho.mqtt.client as mqtt

client = mqtt.Client()
client.connect('broker', 1883, 60)
for i in range(5):
    client.publish('lab/topic', f'Message {i}', qos=1)
    time.sleep(1)
client.disconnect()
```

Bring the stack up:

```
docker compose up --build
```

Deliverable: Provide `docker-compose.yml`, the publisher script, and logs/screenshots showing successful publish/subscribe.